UID-keyed Fingerprint API

Backend reference for the Flutter / frontend team.

Overview

Every person is identified by one **UID**: exactly 12 letters and digits, chosen by the operator or user. Single-finger and slap captures enrol under the **same** UID. Verification is always **1:1** against that one UID — there is no search across people.

- A single-finger probe is matched only against single-finger samples, **at the same finger position**.
- A slap probe is matched only against slap samples, finger by finger at the same position.
- A finger can hold up to **5 samples** (more samples improve the odds of a match). A verify scores the probe against all of that finger's samples and takes the best score.
- A slap verify **passes when at least 2 fingers pass**.

These endpoints are new. The existing `/enroll`, `/authenticate`, `/verify`, `/v2/*` and `*_slap` endpoints are unchanged and do not share data with these.

Base URLs and authentication

Server	Base URL	Used for
Single finger	`http://35.255.138.39:5002`	`/uid/enroll`, `/uid/verify`
Slap	`http://35.255.138.39:5010`	`/uid/enroll_slap`, `/uid/verify_slap`

`/uid/register` and `GET /uid/{uid}` work on **either** server (both share one database).

Every `/uid/*` request must carry the header:

```
X-API-Key: <shared key provided by the backend team>
```

Missing or wrong key returns `401`. If the server has no key configured it returns `503` (`api_key_not_configured`) and refuses to run.

Typical flow

1. `POST /uid/register` — once per person.
2. `POST /uid/enroll` (one call per finger sample) and/or `POST /uid/enroll_slap`. Repeat for the same finger to add more samples (max 5 per finger).
3. `GET /uid/{uid}` — see which fingers are enrolled, so the app can tell the user which finger to present.
4. `POST /uid/verify` or `POST /uid/verify_slap`.
5. Fix mistakes with the delete / rename endpoints below.

UID rules

- Exactly 12 characters, `A-Z` and `0-9` only.
- Case-insensitive: the server uppercases it. `ab12cd34ef56` and `AB12CD34EF56` are the same person.
- Registering an existing UID returns `409`.

Finger positions

- **Single:** integer `1`–`10`, ISO 19794 codes — 1 right thumb, 2 right index, 3 right middle, 4 right ring, 5 right little, 6 left thumb, 7 left index, 8 left middle, 9 left ring, 10 left little.
- **Slap:** names assigned by the server from `hand_side` (default `right`), left to right in the image:
- `right`: `right_little`, `right_ring`, `right_middle`, `right_index`
- `left`: `left_index`, `left_middle`, `left_ring`, `left_little`

The optional form field `finger_order` (comma-separated names) overrides this.

POST /uid/register

Form fields (`multipart/form-data` or urlencoded):

Field	Required	Notes
`uid`	yes	12 letters/digits
`name`	no	

`201`

```
{ "success": true, "uid": "AB12CD34EF56", "name": "Asha", "created_at": "2026-10-15T09:30:00.123456" }
```

Errors: `400 invalid_uid`, `409 uid_exists`.

GET /uid/{uid}

`200`

```
{
  "success": true,
  "uid": "AB12CD34EF56",
  "name": "Asha",
  "created_at": "2026-10-15T09:30:00.123456",
  "enrolled": {
    "single": { "2": 3, "3": 1 },
    "slap":   { "right_index": 2, "right_middle": 2 }
  },
  "max_samples_per_finger": 5
}
```

`enrolled` maps finger position to the number of samples stored. Errors: `404 uid_not_found`.

POST /uid/enroll (single server, :5002)

Adds **one** sample for one finger. Call again to add more samples.

Field	Required	Notes
`uid`	yes	must be registered
`finger_position`	yes	integer 1–10
`image`	yes	finger photo (file)

`200`

```
{
  "success": true, "uid": "AB12CD34EF56", "finger_position": "2",
  "samples_for_finger": 2, "max_samples": 5, "minutiae_count": 38,
  "detection_conf": 0.9123, "liveness": { "is_live": true, "confidence": 0.98 }
}
```

POST /uid/verify (single server, :5002)

Field	Required	Notes
`uid`	yes	
`finger_position`	yes	the finger being presented
`image`	yes	

`200`

```
{
  "success": true, "uid": "AB12CD34EF56", "finger_position": "2",
  "matched": true, "score": 0.81234, "threshold": 0.4,
  "samples_compared": 3, "input_minutiae_count": 41
}
```

`matched: false` is a normal `200` response (the finger did not match), not an error.

POST /uid/enroll_slap (slap server, :5010)

Adds **one sample per detected finger** from a single slap photo.

Field	Required	Notes
`uid`	yes	must be registered
`image`	yes	slap photo
`hand_side`	no	`left` or `right` (default `right`)
`finger_order`	no	comma-separated finger names

`200`

```
{
  "success": true, "uid": "AB12CD34EF56", "max_samples": 5,
  "enrolled_fingers": [
    { "finger_position": "right_index", "samples_for_finger": 1, "minutiae_count": 36 },
    { "finger_position": "right_middle", "samples_for_finger": 1, "minutiae_count": 40 }
  ],
  "skipped_fingers": ["right_little"]
}
```

`skipped_fingers` lists fingers that were detected but unusable (too few minutiae or a processing failure). The enrolment is **all-or-nothing**: if any finger is already at 5 samples the whole request fails with `409 sample_cap_reached` and nothing is stored.

POST /uid/verify_slap (slap server, :5010)

Same fields as `enroll_slap`.

`200`

```
{
  "success": true, "uid": "AB12CD34EF56", "matched": true,
  "fingers_passed": 3, "fingers_required": 2, "threshold": 0.4,
  "fingers": [
    { "finger_position": "right_index", "enrolled": true,
      "score": 0.83, "passed": true, "samples_compared": 2 },
    { "finger_position": "right_middle", "enrolled": true,
      "score": 0.61, "passed": true, "samples_compared": 2 },
    { "finger_position": "right_ring", "enrolled": true,
      "score": 0.12, "passed": false, "samples_compared": 1 },
    { "finger_position": "right_little", "enrolled": false,
      "score": null, "passed": false }
  ],
  "skipped_fingers": []
}
```

`matched` is true when `fingers_passed >= fingers_required`.

PATCH /uid/{uid}

Renames the person. Nothing else about a person can be edited; to change a template, delete the finger and enrol it again.

Field	Required	Notes
`name`	yes	non-empty

`200`: same body as `GET /uid/{uid}`. Errors: `400 bad_request`, `404 uid_not_found`.

DELETE /uid/{uid}/samples

Deletes **every sample** of one finger, so that finger can be enrolled again from scratch (this also frees its 5-sample cap).

Query (or form) parameters: `modality` (`single` or `slap`) and `finger_position`.

Example: `DELETE /uid/AB12CD34EF56/samples?modality=single&finger_position=2`

`200`

```
{ "success": true, "uid": "AB12CD34EF56", "deleted_samples": 3 }
```

Errors: `404 uid_not_found`, `409 finger_not_enrolled` (nothing to delete), `400 bad_request`.

DELETE /uid/{uid}?confirm=true

Permanent. Deletes the person, all their templates and their verification log. The UID can be registered again afterwards. Without `confirm=true` the call is refused with `400`, so a stray request cannot wipe someone.

`200`

```
{ "success": true, "uid": "AB12CD34EF56", "deleted_samples": 7 }
```

Errors: `400 bad_request` (missing confirm), `404 uid_not_found`.

Errors

Error bodies look like:

```
{ "success": false, "error": "uid_not_found", "message": "uid not registered" }
```

HTTP	`error`	Meaning / suggested app behaviour
400	`invalid_uid`	UID is not 12 letters/digits
400	`bad_request`	e.g. `finger_position` out of range or missing
400	`image required`	no `image` file in the request
401	`unauthorized`	missing / wrong `X-API-Key`
404	`uid_not_found`	UID was never registered
409	`uid_exists`	UID already registered
409	`sample_cap_reached`	finger already has 5 samples (`fingers` and `max_samples` in body)
409	`finger_not_enrolled`	single verify: no sample for that finger — enrol it first; also returned when deleting a finger that has no samples

HTTP	`error`	Meaning / suggested app behaviour
409	`not_enough_enrolled_fingers`	slap verify: UID has fewer than 2 enrolled slap fingers (`enrolled_fingers`, `required` in body)
422	— with `quality_failed: true`	photo failed quality check; show `guidance` and retake
422	— with `spoof_detected: true`	liveness / screen-replay check failed
422	`Too few minutiae — retake`	not enough ridge detail; retake
503	`api_key_not_configured`	server misconfiguration; contact the backend team
500	`error` string	unexpected server error

Deploying on the existing VM (`35.255.138.39`)

The `/uid/*` routes are additive: the old endpoints and databases keep working unchanged.

1. Pull this branch onto the VM.
2. Set the environment for both servers: `API_KEY=<secret>` (and optionally the `UID_*` variables below). Both servers must see the same `IDENTITY_DB` file; the default does this when both run from the same `backend/` folder.
3. Restart the servers on :5002 (`app.py`) and :5010 (`slap_app.py`). `identity.db` is created on first start.
4. Smoke test: register a UID, enrol a finger photo, verify with a second photo of the same finger, then `GET /uid/{uid}`.
5. Give the frontend team the API key and this document.

Notes for the backend / deployment team

Environment variables on the VM (all read at request time except `API_KEY` which must exist):

Variable	Default	Meaning
`API_KEY`	— (required)	shared secret for `X-API-Key`
`UID_MATCH_THRESH`	`0.40`	per-finger pass threshold
`UID_MIN_FINGERS_PASS`	`2`	slap fingers that must pass
`UID_MAX_SAMPLES`	`5`	samples kept per finger
`UID_MATCH_ALGORITHM`	`ransac_alignment`	scorer from `matching.py`
`IDENTITY_DB`	`backend/identity.db`	SQLite file shared by both servers

The threshold and algorithm are uncalibrated. They were chosen from synthetic data only and must be recalibrated from a measured FAR/FRR curve on real captures before the accuracy gate is claimed. Both servers must point at the same `IDENTITY_DB` file (the default does this when both run from the same `backend/` folder).